

**VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY**



**Digital Systems
Lab 5**

Instructor: Đoàn Ngọc Cẩm

Student	Student ID
Hồ Chí Dũng	2351016
Nguyễn Viết Thi	2451165
Trần Duy Anh	2351007

Ho Chi Minh City – May 2026

Laboratory 5:

AN ENHANCED PROCESSOR

OBJECTIVES

- The purpose of this lab is to learn how to connect simple input (switches) and output devices (LEDs and 7-segment) to an FPGA chip and implement a circuit that uses these devices.
- Design an enhanced processor based on the simple processor in lab 4.

PREPARATION FOR LAB 5

- Finish Pre Lab 5 at home.
- Students have to simulate all the exercises in Pre Lab 5 at home. All results (codes, waveform, RTL viewer, ...) have to be captured and submitted to instructors prior to the lab session.
If not, students will not participate in the lab and be considered absent this session.

REFERENCE

1. Intel FPGA training



Laboratory 5:

AN ENHANCED PROCESSOR

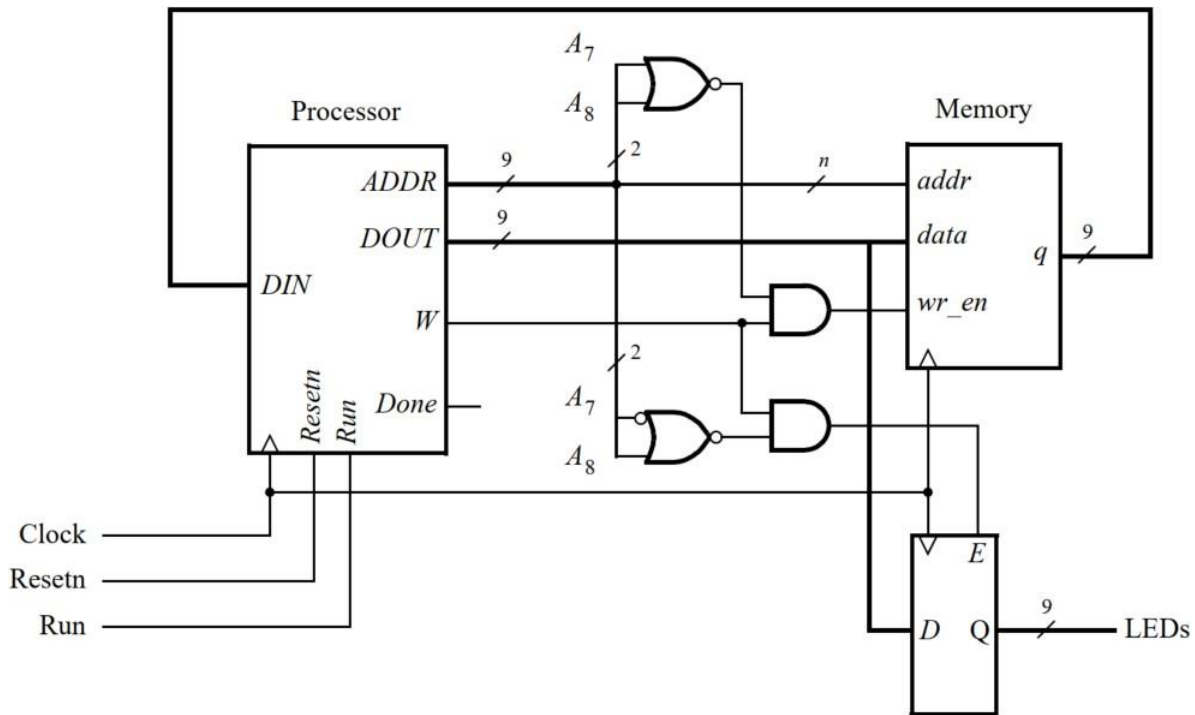


Figure 2: Connecting the enhanced processor to a memory and output register.

Instruction:

- Create a new Quartus project for the enhanced version of the processor.
- Write System Verilog code for the processor and test your circuit by using functional simulation: apply instructions to the DIN port and observe the internal processor signals as the instructions are executed. Pay careful attention to the timing of signals between your processor and external memory; account for the fact that the memory has registered input ports, as we discussed for Figure 2.
- Create another Quartus project that instantiates the processor, memory module, and register shown in Figure 2. Use the Quartus IP Catalog to create the RAM: 1-PART memory module. Follow the instructions provided by the wizard to create a memory that has one 9-bit wide read/write data port and is 128 words deep. Ensure that the output is not registered. Use a MIF file to store instructions in the memory that are to be executed by your processor. An example program in the form of a MIF file is shown in Figure 3. This program display an 8-bit counter value on the



Laboratory 5:

AN ENHANCED PROCESSOR

LEDs output port. Loops are used in the program to create delays so that the counter values are not changed too quickly to be observed. Comments are included in the MIF file in Figure 3 to describe the program's code.

```

DEPTH = 128;
WIDTH = 9;
ADDRESS_RADIX = HEX;
DATA_RADIX = BIN;
CONTENT
BEGIN

% This code displays a count (in register R2) on the red LEDs.
00 : 001001000; %      mvi    R1,#1           // initialize R
01 : 000000001;
02 : 001010000; %      mvi    R2,#0           // counter to display on LEDs
03 : 000000000;
04 : 001011000; % Loop  mvi    R3,#010000000    // R3 = address of LEDs register
05 : 010000000;
06 : 101010011; %      st     R2,R3           // write to LEDs
07 : 010010001; %      add    R2,R1           // increment counter for LEDs
08 : 001011000; %      mvi    R3,#111111111    // delay value
09 : 111111111;
0A : 000101111; %      mv     R5,R7           // save address of next inst.
0B : 001100000; % Outer  mvi    R4,#111111111    // nested delay loop
0C : 111111111;
0D : 000000111; %      mv     R0,R7           // save address of next inst.
0E : 011100001; % Inner  sub    R4,R1           // decrement loop delay variable
0F : 110111000; %      mvnz   R7,R0           // continue Inner loop if R4 !=0
10 : 011011001; %      sub    R3,R1           // decrement outer loop delay
11 : 110111101; %      mvnz   R7,R5           // continue Outer loop if R3 !=0
12 : 001111000; %      mvi    R7,#Loop        // execute again
13 : 000000100;

END;
```

➤ Figure 3: An example program in a memory initialization file (MIF).

➤ Use functional simulation to test the circuit. Ensure that data is read properly from the memory and executed by the processor

➤ Include in your project the necessary pin assignments to implement your circuit on your DE-series board. Use switch **SW9** to drive the processor's Run input, use KEY0 for Resetn, and



Laboratory 5:

AN ENHANCED PROCESSOR

use the board's 50 MHz clock signal as the Clock input. Since the circuit needs to run properly at 50 MHz, make sure that a timing constraint is set in Quartus to constrain the circuit's clock to this frequency. Read the Report produced by the Quartus Timing Analyzer to ensure that your circuit operates at this speed; if not, use the Quartus tools to analyze your circuit and modify your Verilog code to make a more efficient design that meets the 50-MHz speed requirement. Also note that the Run input is asynchronous to the clock signal, so make sure to synchronize this input using flip-flops. Connect the LEDs register in Figure 2 to LEDR8–0 so that you can observe the output produced by the processor.

➤ Compile the circuit, download it into the FPGA chip, and ensure that your program runs properly.

Check: Your report has to show two results:

➤ The waveform to prove the circuit works correctly.

➤ The result of RTL viewer.

CODE:

```
module DE10 (
    input logic      CLOCK_50, // Xung clock 50MHz của mạch
    input logic [0:0] KEY,      // KEY[0] dùng làm nút bấm Resetn
    input logic [9:9] SW,       // SW[9] dùng làm công tắc Run
    output logic [8:0] LEDR      // LEDR[8:0] nối ra 9 bóng đèn LED đỏ
);

// --- 1. KHAI BÁO DÂY NỐI NỘI BỘ ---
logic [8:0] ADDR;
logic [8:0] DOUT;
logic [8:0] DIN;
logic      W;
logic      Done;
logic [8:0] BusWires; // Giữ lại để debug nếu cần

// Tín hiệu giải mã địa chỉ (Address Decoding)
logic ram_wr_en;
logic led_en;

// --- 2. MẠCH GIẢI MÃ ĐỊA CHỈ ---
// RAM được ghi khi 2 bit địa chỉ cao nhất A8, A7 = 00 VÀ có cờ Write (W=1)
assign ram_wr_en = ~(ADDR[8] | ADDR[7]) & W;

// LED được cập nhật khi A8, A7 = 01 VÀ có cờ Write (W=1)
// Phép AND ở cổng dưới cùng của Hình 2: A7 đưa thẳng vào, A8 qua cổng NOT
assign led_en    = ~(ADDR[8] | ~ADDR[7]) & W;
```



Laboratory 5:

AN ENHANCED PROCESSOR

```
// --- 3. CẤM CÁC THÀNH PHẦN VÀO HỆ THỐNG ---

// A. Lỗi Vi xử lý (Processor)
lab5 cpu (
    .DIN(DIN),
    .Resetn(KEY[0]),
    .Clock(CLOCK_50),
    .Run(SW[9]),
    .Done(Done),
    .BusWires(BusWires),
    .ADDR(ADDR),
    .DOUT(DOUT),
    .W(W)
);

// B. Bộ nhớ RAM (Memory 128 words x 9 bits)
// Lưu ý: Module này KHÔNG tự có sẵn, bạn phải tạo bằng Quartus IP Catalog!
my_ram ram (
    .address (ADDR[6:0]), // Chỉ lấy 7 bit thấp (A6 -> A0) để quét 128 ô
    .clock   (CLOCK_50), // RAM đồng bộ dùng chung xung clock
    .data    (DOUT),      // Dữ liệu từ CPU đẩy ra chờ ghi vào RAM
    .wren    (ram_wr_en), // Cờ báo hiệu ghi vào RAM
    .q       (DIN)        // Dữ liệu RAM nhả ra để CPU đọc (lệnh hoặc data)
);

// C. Thanh ghi LED (Output Register)
always_ff @(posedge CLOCK_50 or negedge KEY[0]) begin
    if (!KEY[0]) begin
        LEDR <= 9'b00000000; // Reset tắt hết bóng đèn
    end else if (led_en) begin
        LEDR <= DOUT;        // Chốt dữ liệu từ DOUT ra đèn LED
    end
end

endmodule

module my_ram #(
    parameter DATA_WIDTH = 9,
    parameter ADDR_WIDTH = 7
) (
    input logic clock,
    input logic wren,
    input logic [ADDR_WIDTH-1:0] address,
    input logic [DATA_WIDTH-1:0] data,
    output logic [DATA_WIDTH-1:0] q
);

//(* ram_init_file = "lab5.mif" *)
logic [8:0] ram_block [0:127];

initial begin
```



Laboratory 5:

AN ENHANCED PROCESSOR

```
$readmemb("lab5.txt", ram_block);
end

always_ff @(posedge clock) begin
    if (wren) begin
        ram_block[address] <= data;
    end
    // Luôn luôn đọc dữ liệu ra
    q <= ram_block[address];
end

endmodule
// --- MODULE CHÍNH (Processor) ---
module lab5 #(parameter n = 9) (
    input logic [8:0] DIN,
    input logic      Resetn, Clock, Run,
    output logic     Done,
    output logic [8:0] BusWires, // Giữ lại đây để xem waveform
    output logic [n-1:0] ADDR,
    output logic [n-1:0] DOUT,
    output logic      W
);
    // Khai báo TẤT CẢ dây nối nội bộ lên TRÊN CÙNG
    logic [8:0] IR;
    logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7, A, G;
    logic [8:0] AddSub_Result;
    logic      IRin, DINout, Ain, Gin, Gout, AddSub;
    logic [7:0] Rin, Rout;
    logic      ADDRin, DOUTin, W_D, incr_pc;
    logic      G_not_zero;

    // Logic kiểm tra G
    assign G_not_zero = (G != 9'b00000000);

    // Thêm mạch đồng bộ 2 tầng cho chân Run
    logic run_sync_1, run_sync_2;
    always_ff @(posedge Clock or negedge Resetn) begin
        if (!Resetn) begin
            run_sync_1 <= 1'b0;
            run_sync_2 <= 1'b0;
        end else begin
            run_sync_1 <= Run;
            run_sync_2 <= run_sync_1;
        end
    end

    // 1. Cắm khối Điều khiển (Control Unit)
    control_unit CU (
        .Clock(Clock),
        .Resetn(Resetn),
        .Run(run_sync_2),
        .IR(IR),
        .IRin(IRin),
        .DINout(DINout),
```



Laboratory 5:

AN ENHANCED PROCESSOR

```

        .Ain(Ain),
        .Gin(Gin),
    .Gout(Gout),
        .AddSub(AddSub),
        .Done(Done),
    .Rin(Rin),
        .Rout(Rout),
    .ADDRin(ADDRin),
    .DOUTin(DOUTin),
    .W_D(W_D),
    .G_not_zero(G_not_zero),
    .incr_pc(incr_pc) // Đã sửa lỗi chính tả incrpc
);

// 2. Cắm thanh ghi IR
regn reg_IR (.R(DIN), .Rin(IRin), .Resetn(Resetn), .Clock(Clock), .Q(IR));

// 3. Cắm 8 thanh ghi đa dụng R0 -> R7
regn reg_0 (.R(BusWires), .Rin(Rin[0]), .Resetn(Resetn), .Clock(Clock),
.Q(R0));
regn reg_1 (.R(BusWires), .Rin(Rin[1]), .Resetn(Resetn), .Clock(Clock),
.Q(R1));
regn reg_2 (.R(BusWires), .Rin(Rin[2]), .Resetn(Resetn), .Clock(Clock),
.Q(R2));
regn reg_3 (.R(BusWires), .Rin(Rin[3]), .Resetn(Resetn), .Clock(Clock),
.Q(R3));
regn reg_4 (.R(BusWires), .Rin(Rin[4]), .Resetn(Resetn), .Clock(Clock),
.Q(R4));
regn reg_5 (.R(BusWires), .Rin(Rin[5]), .Resetn(Resetn), .Clock(Clock),
.Q(R5));
regn reg_6 (.R(BusWires), .Rin(Rin[6]), .Resetn(Resetn), .Clock(Clock),
.Q(R6));

pc_register pc_reg_7 (
    .R(BusWires),
    .R7in(Rin[7]),
    .incr_pc(incr_pc), // count wire
    .Clock(Clock),
    .Resetn(Resetn),
    .Q(R7)
);

// Thanh ghi ADDR: Chốt địa chỉ từ Bus ra ngoài
regn reg_ADDR (
    .R(BusWires),
    .Rin(ADDRin),
    .Resetn(Resetn),
    .Clock(Clock),
    .Q(ADDR)
);

```



Laboratory 5:

AN ENHANCED PROCESSOR

```
// Thanh ghi DOUT: Chốt dữ liệu từ Bus ra ngoài
regn reg_DOUT (
    .R(BusWires),
    .Rin(DOUTin),
    .Resetn(Resetn),
    .Clock(Clock),
    .Q(DOUT)
);

// Flip-flop W: Chốt cờ Write (1 bit)
always_ff @(posedge Clock or negedge Resetn) begin
    if (!Resetn)
        W <= 1'b0; // Vừa bật máy là không cho ghi lung tung
    else
        W <= W_D; // Lấy lệnh từ khối FSM cất vào
end

// 4. Cắm thanh ghi A và G
regn reg_A (.R(BusWires), .Rin(Ain), .Resetn(Resetn), .Clock(Clock),
.Q(A));
regn reg_G (.R(AddSub_Result), .Rin(Gin), .Resetn(Resetn), .Clock(Clock),
.Q(G));

// 5. Cắm bộ Tính toán (ALU)
alu ALU_unit (
    .A(A), .BusWires(BusWires), .AddSub(AddSub), .Result(AddSub_Result)
);

// 6. Cắm bộ Dồn kênh (Multiplexer) để tạo ra Bus
multiplexer MUX_unit (
    .R0(R0), .R1(R1), .R2(R2), .R3(R3), .R4(R4), .R5(R5), .R6(R6), .R7(R7),
    .G(G), .DIN(DIN),
    .Rout(Rout), .Gout(Gout), .DINout(DINout),
    .BusWires(BusWires)
);

endmodule

// --- MODULE PROGRAM COUNTER (R7) ---
module pc_register #(parameter n = 9) (
    input logic [n-1:0] R, // Dữ liệu từ Bus (để nạp địa chỉ khi
nhảy/lặp)
    input logic R7in, // Cờ cho phép nạp dữ liệu từ Bus vào R7
    input logic incr_pc, // Cờ cho phép R7 tự đếm lên 1
    input logic Clock, Resetn,
    output logic [n-1:0] Q // Đầu ra (chứa địa chỉ hiện tại)
);
always_ff @(posedge Clock or negedge Resetn) begin
    if (!Resetn)
        Q <= '0; // Mới bật máy lên, PC tự động chỉ về dòng
số 0
    else if (R7in)
```



Laboratory 5:

AN ENHANCED PROCESSOR

```

        Q <= R;                                // Ưu tiên 1: Lệnh nhảy! Nạp địa chỉ mới từ
Bus vào
        else if (incr_pc)
            Q <= Q + 1'b1;                      // Ưu tiên 2: Không có lệnh nhảy thì tự động
nhích xuống 1 dòng
        end
    endmodule

// --- MODULE GIẢI MÃ ĐÃ SỬA LỖI NGƯỢC BIT ---
module dec3to8(
    input logic [2:0] W,
    input logic      En,
    output logic [7:0] Y
);
    always_comb begin
        if (En == 1)
            case (W)
                3'b000: Y = 8'b00000001;
                3'b001: Y = 8'b00000010;
                3'b010: Y = 8'b00000100;
                3'b011: Y = 8'b00001000;
                3'b100: Y = 8'b00010000;
                3'b101: Y = 8'b00100000;
                3'b110: Y = 8'b01000000;
                3'b111: Y = 8'b10000000;
            endcase
        else Y = 8'b00000000;
    end
endmodule

// --- MODULE THANH GHI (Register) ---
module regn #(parameter n = 9) (
    input logic [n-1:0] R,
    input logic      Rin, Clock, Resetn,
    output logic [n-1:0] Q
);
    always_ff @(posedge Clock or negedge Resetn)
        if (!Resetn)
            Q <= '0;
        else if (Rin)
            Q <= R;
    endmodule

// --- MODULE ALU ---
module alu (
    input logic [8:0] A,
    input logic [8:0] BusWires,
    input logic      AddSub,
    output logic [8:0] Result
);
    always_comb begin
        if (AddSub)
            Result = A - BusWires; // AddSub = 1 thì trừ
    end
endmodule

```



Laboratory 5:

AN ENHANCED PROCESSOR

```

        else
            Result = A + BusWires; // AddSub = 0 thì cộng
        end
    endmodule

// --- MODULE MULTIPLEXER ---
module multiplexer (
    input logic [8:0] R0, R1, R2, R3, R4, R5, R6, R7,
    input logic [8:0] G, DIN,
    input logic [7:0] Rout,
    input logic      Gout, DINout,
    output logic [8:0] BusWires
);
    always_comb begin
        BusWires = 9'b000000000;
        //BusWires = 9'b0; // Mặc định để tránh sinh ra Latch
        if (DINout)      BusWires = DIN;
        else if (Gout)   BusWires = G;
        else if (Rout[0]) BusWires = R0;
        else if (Rout[1]) BusWires = R1;
        else if (Rout[2]) BusWires = R2;
        else if (Rout[3]) BusWires = R3;
        else if (Rout[4]) BusWires = R4;
        else if (Rout[5]) BusWires = R5;
        else if (Rout[6]) BusWires = R6;
        else if (Rout[7]) BusWires = R7;
    end
endmodule

//---CONTROL UNIT---
module control_unit (
    input logic      Clock, Resetn, Run,
    input logic [8:0] IR,
    input logic      G_not_zero, // NEW: Cờ báo G != 0 (từ bên ngoài
truyền vào)
    output logic      IRin, DINout, Ain, Gin, Gout, AddSub, Done,
    output logic [7:0] Rin, Rout,
    // NEW
    output logic      ADDRin, DOUTin, W_D, incr_pc
);
    //Tăng lên 3-bit để chứa đủ 7 trạng thái (T0 đến T6)
    typedef enum logic [2:0] {T0=3'd0, T1=3'd1, T2=3'd2, T3=3'd3, T4=3'd4,
T5=3'd5, T6=3'd6} state_t;
    state_t Tstep_Q, Tstep_D;

    // Phân rã mã lệnh
    logic [2:0] I;
    logic [7:0] Xreg, Yreg;
    assign I = IR[8:6];

    // Gọi module Decoder giải mã Rx và Ry
    dec3to8 decX (.W(IR[5:3]), .En(1'b1), .Y(Xreg));
    dec3to8 decY (.W(IR[2:0]), .En(1'b1), .Y(Yreg));

```



Laboratory 5:

AN ENHANCED PROCESSOR

```

// FSM State Register
always_ff @(posedge Clock, negedge Resetn) begin
    if (!Resetn) Tstep_Q <= T0;
    else          Tstep_Q <= Tstep_D;
end

// FSM Next State Logic
always_comb begin
    case (Tstep_Q)
        T0: if (!Run) Tstep_D = T0; else Tstep_D = T1;
        T1: Tstep_D = T2;
        T2: Tstep_D = T3;
        T3: if (Done) Tstep_D = T0; else Tstep_D = T4;
        T4: if (Done) Tstep_D = T0; else Tstep_D = T5;
        T5: if (Done) Tstep_D = T0; else Tstep_D = T6;
        T6: Tstep_D = T0;
        default: Tstep_D = T0;
    endcase
end

// FSM Output Logic
always_comb begin
    // Khởi tạo mặc định
    IRin = 0; DINout = 0; Ain = 0; Gin = 0; Gout = 0;
    AddSub = 0; Rin = 8'b0; Rout = 8'b0; Done = 0;
    ADDRin = 0; DOUTin = 0; W_D = 0; incr_pc = 0;

case (Tstep_Q)

    // -----
    // QUY TRÌNH ĐỌC LỆNH (FETCH) - 3 NHỊP
    // -----
    T0: begin
        Rout[7] = 1'b1; // Mở cửa R7 (PC) đẩy địa chỉ ra Bus
        ADDRin = 1'b1; // Chốt vào ADDR gửi cho RAM
    end
    T1: begin
        // NHỊP CHỜ (WAIT STATE): Để cục RAM đồng bộ kịp nuốt địa chỉ
    end
    T2: begin
        IRin = 1'b1; // RAM nhà dữ liệu, hút mã lệnh vào IR
        incr_pc = 1'b1; // Lấy xong thì PC đếm lên 1
    end

    // -----
    // QUY TRÌNH THỰC THI (EXECUTE)
    // -----
    T3: begin
        case (I)
            3'b000: begin Rout = Yreg; Rin = Xreg; Done = 1; end
            3'b001: begin Rout[7] = 1'b1; ADDRin = 1'b1; end
            // mvi: Đưa PC ra ADDR để lấy Dữ liệu

```



Laboratory 5:

AN ENHANCED PROCESSOR

```

        3'b010, 3'b011: begin Rout = Xreg; Ain = 1; end
// add, sub: Đưa Rx vào A
        3'b100: begin Rout = Yreg; ADDRin = 1'b1; end
// ld Rx, [Ry]: Đưa Ry (địa chỉ) ra ADDR
        3'b101: begin Rout = Yreg; ADDRin = 1'b1; end
// st Rx, [Ry]: Đưa Ry (địa chỉ) ra ADDR
        3'b110: begin // mvnz Rz, Ry
            if (G_not_zero) begin
                Rout = Yreg;
                Rin = Xreg;
// Bộ giải mã sẽ tự lo việc bật đúng R7in
            end
            Done = 1;
        end
    endcase
end

T4: begin
    case (I)
        3'b001: ; // mvi: NHỊP CHỜ RAM
        3'b010: begin Rout = Yreg; Gin = 1; AddSub = 0; end // add
        3'b011: begin Rout = Yreg; Gin = 1; AddSub = 1; end // sub
        3'b100: ; // ld: NHỊP CHỜ RAM
        3'b101: begin Rout = Xreg; DOUTin = 1'b1; end
// st: Đưa Rx (dữ liệu) vào DOUT
    endcase
end

T5: begin
    case (I)
        3'b001: begin DINout = 1; Rin = Xreg; incr_pc = 1; Done = 1;
end // mvi: Hút Data, tăng pc lần 2!
        3'b010, 3'b011: begin Gout = 1; Rin = Xreg; Done = 1; end
// add, sub
        3'b100: begin DINout = 1; Rin = Xreg; Done = 1; end
// ld: Hút Data từ RAM vào Rx
        3'b101: begin W_D = 1'b1; end
// st: Bật cờ W lên 1
    endcase
end

T6: begin
    case (I)
// st: Kéo dài thêm 1 nhịp để RAM kịp thấy cờ W=1 rồi mới kết thúc lệnh
        3'b101: begin Done = 1; end
    endcase
end
endcase
end
endmodule

```



Laboratory 5:

AN ENHANCED PROCESSOR

TXT FILE CONTAIN INSTRUCTION CODE:

```
001001000
000000001
001010000
000000000
001011000
010000000
101010011
010010001
001011000
000000010 //delay: 000000010 for stimulation, 111111111 for upload on DE10 KIT
000101111
001100000
000000010 //delay: 000000010 for stimulation, 111111111 for upload on DE10 KIT
000000111
011100001
110111000
011011001
110111101
001111000
000000100
```

TEST BENCH CODE:

Remember to set the delay in txt file, so that it is appropriate (Ex: 000 000 010)

```
`timescale 1ns/1ps

module DE10_tb;

    // 1. Khai báo các tín hiệu ngoại vi của board DE10
    logic        CLOCK_50;
    logic [0:0]  KEY;
    logic [9:9]  SW;
    logic [8:0]  LEDR;

    // 2. Gọi module Top-level (DUT)
    DE10 dut (
        .CLOCK_50(CLOCK_50),
        .KEY(KEY),
        .SW(SW),
        .LEDR(LEDR)
    );

    // 3. Tạo xung Clock 50MHz (Chu kỳ 20ns)
    initial begin
        CLOCK_50 = 0;
        forever #10 CLOCK_50 = ~CLOCK_50;
    end

    // 4. Kích bản mô phỏng
    initial begin
```



Laboratory 5:

AN ENHANCED PROCESSOR

```
// Trạng thái tắt máy
KEY[0] = 0; // Đè nút Reset
SW[9] = 0; // Tắt công tắc Run
#50;

// Nhả nút Reset
KEY[0] = 1;
#50;

// Gạt công tắc Run lên để bắt đầu chạy chương trình trong RAM
SW[9] = 1;

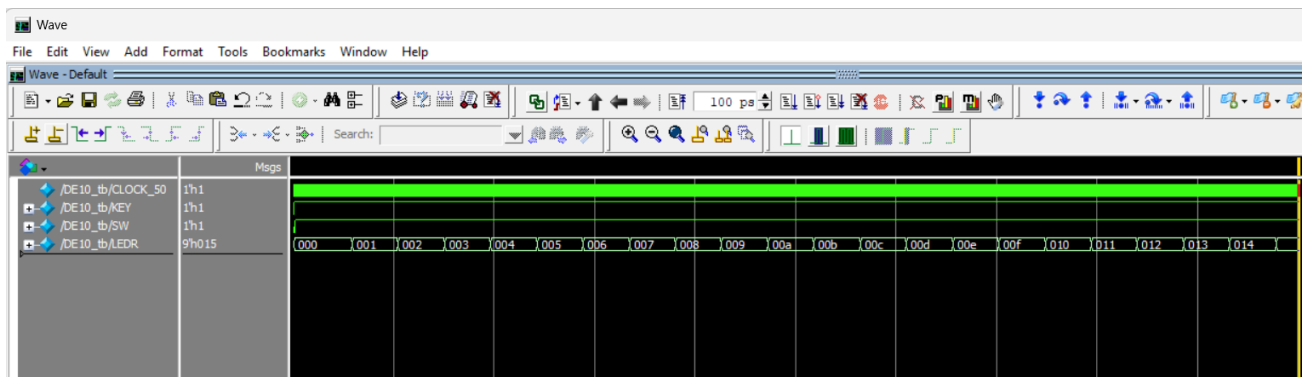
// CHÚ Ý: Chạy mô phỏng trong thời gian DÀI (50,000 ns)
// Để CPU có thời gian khởi tạo và chạy qua các vòng lặp trễ
#50000;

$stop;

end

endmodule
```

WAVEFORM:

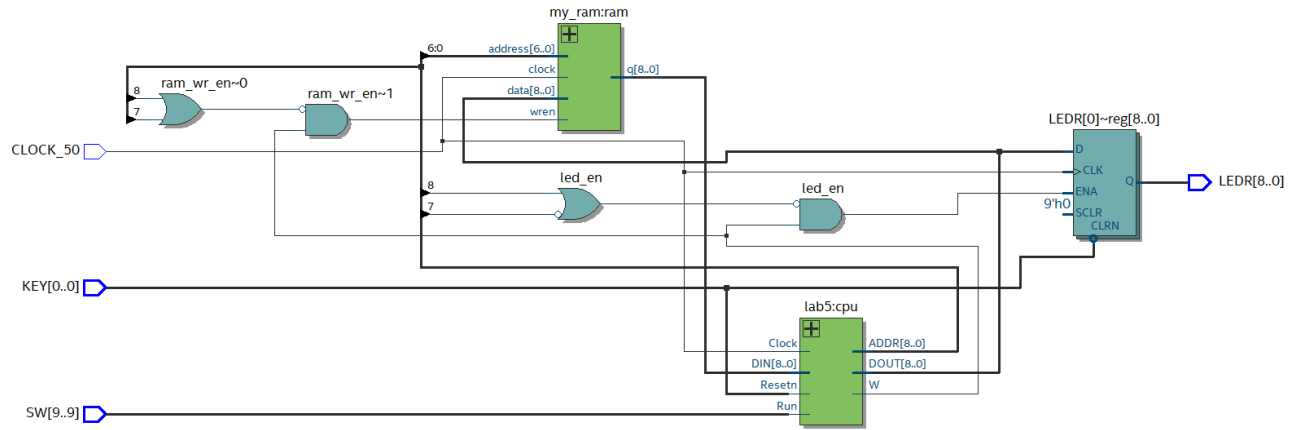


Because we set the time stimulation only #50000 ns. So, it only count to 15. If we set time larger, then it will count more.

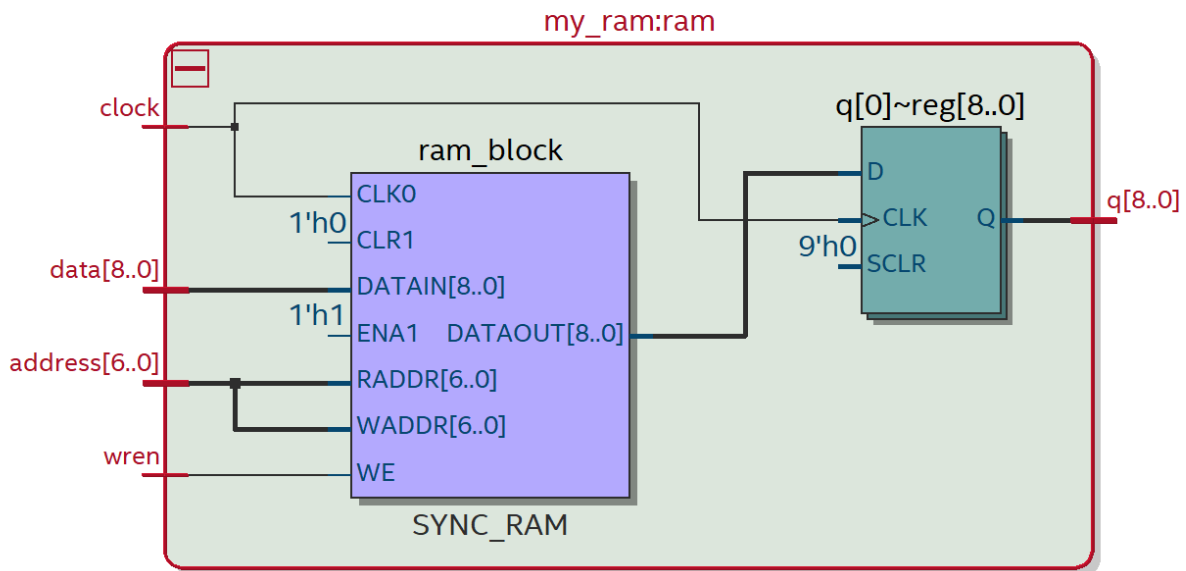


Laboratory 5: AN ENHANCED PROCESSOR

RTL VIEWER:

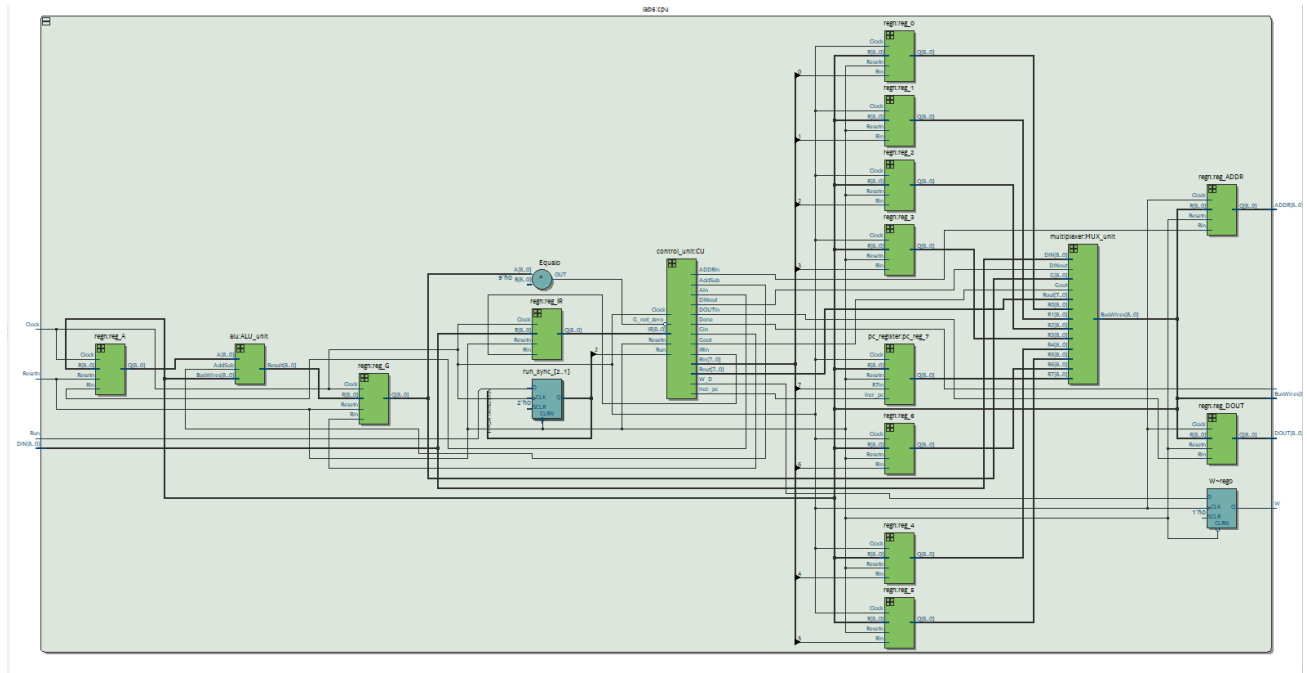


my_ram:

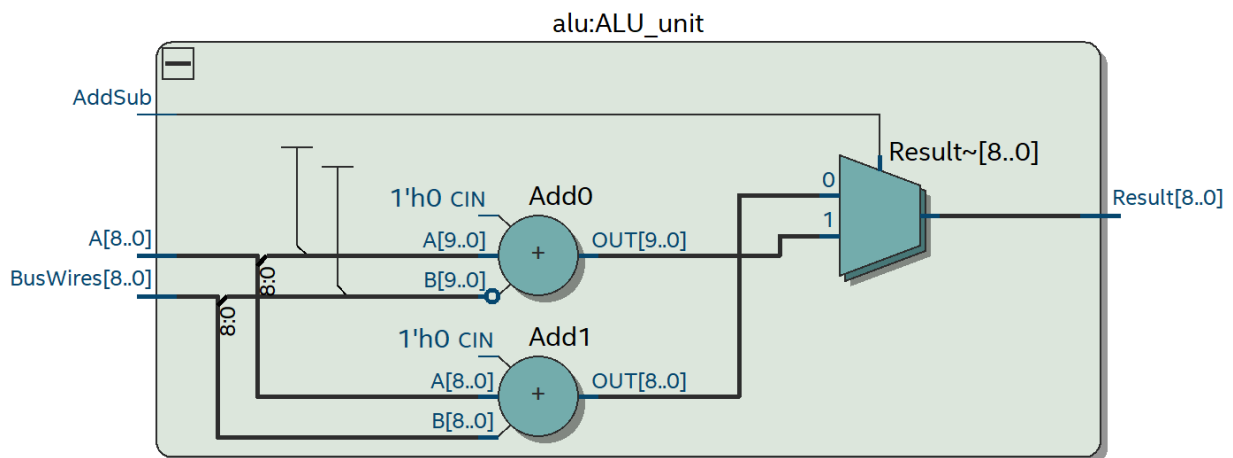


Laboratory 5: AN ENHANCED PROCESSOR

lab5 cpu:



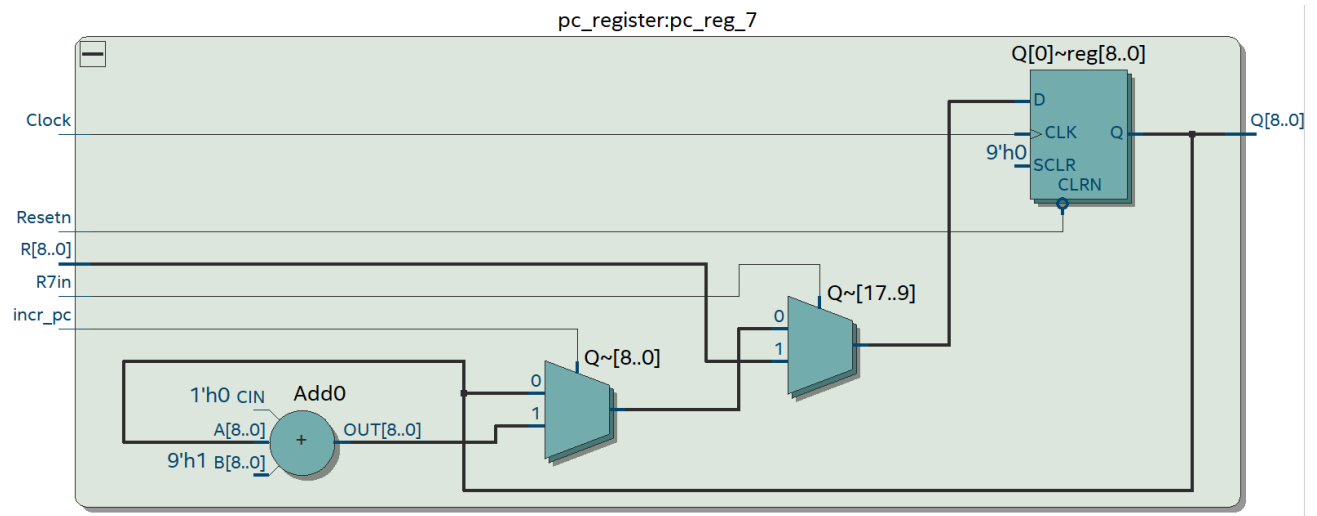
ALU:



Laboratory 5:

AN ENHANCED PROCESSOR

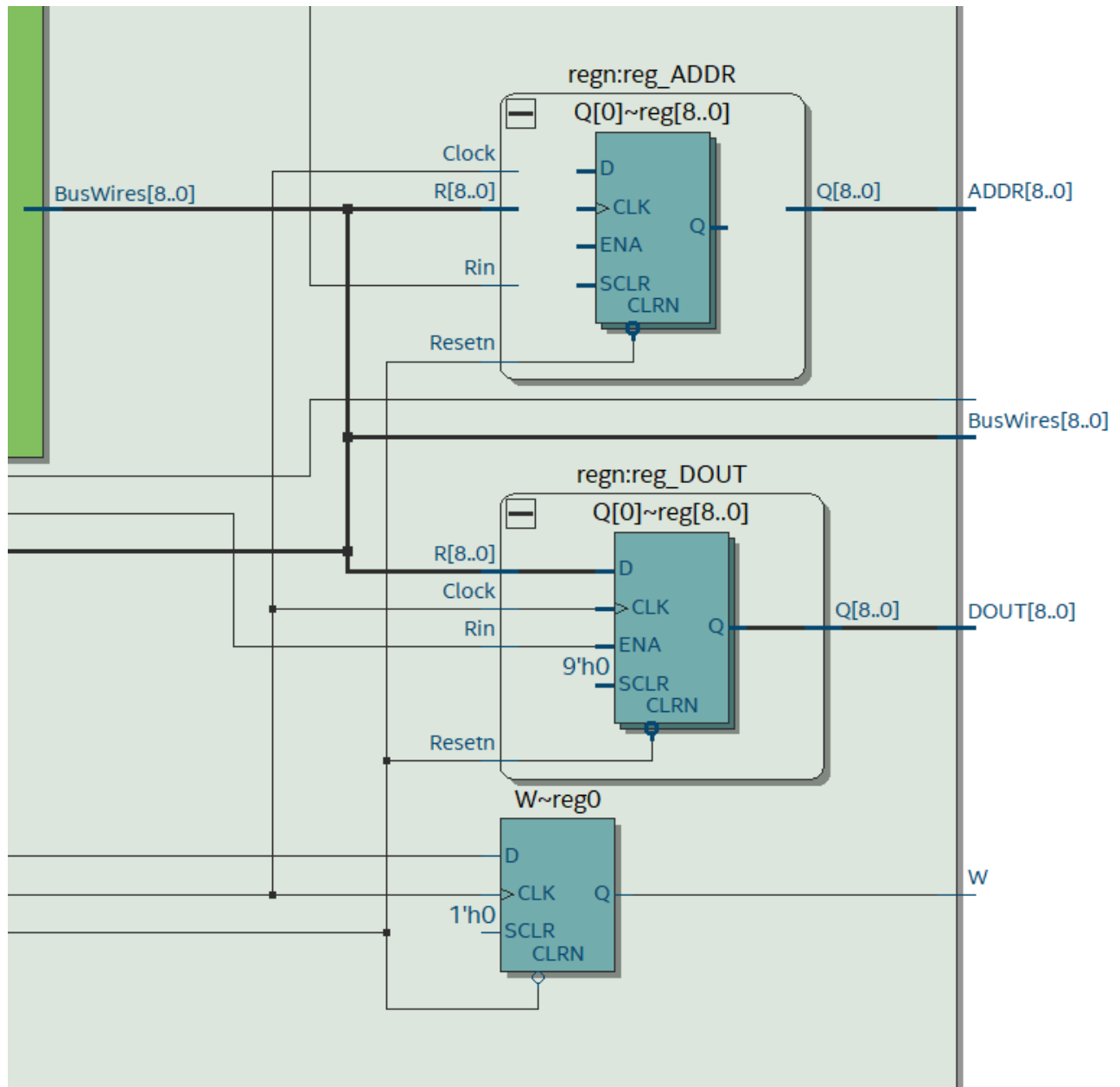
REG7:



Laboratory 5:

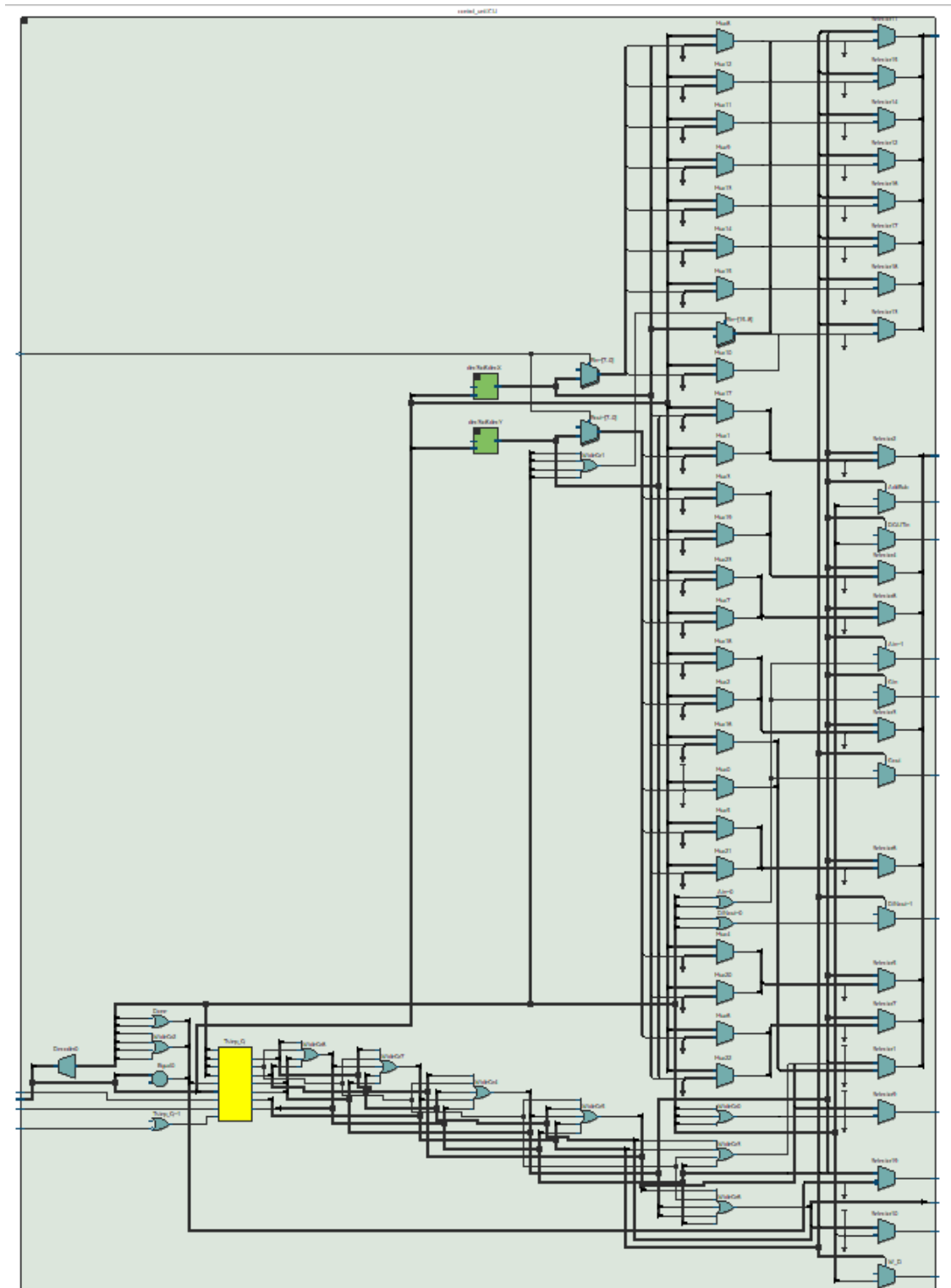
AN ENHANCED PROCESSOR

Reg ADDR, Reg DOUT and W



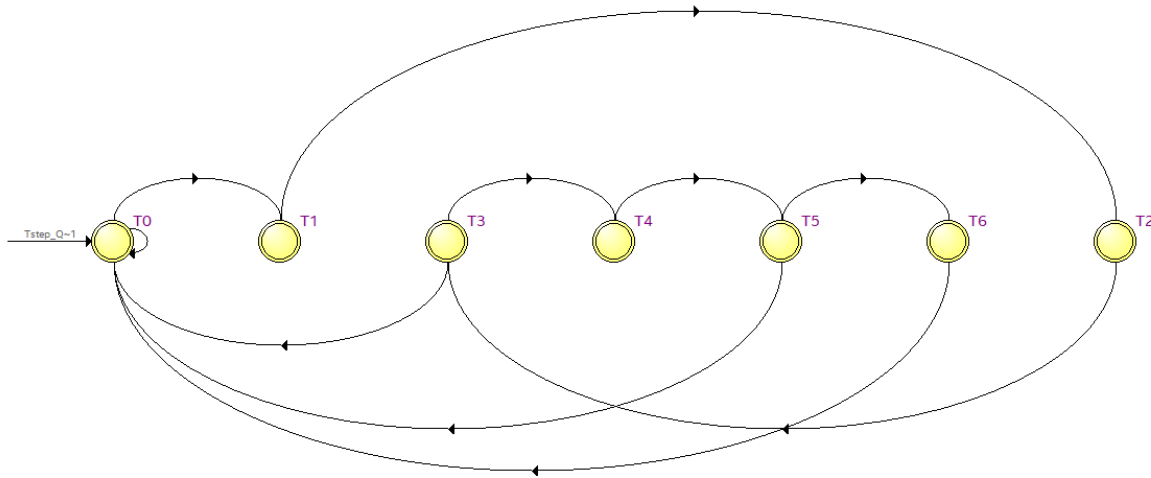
Laboratory 5: AN ENHANCED PROCESSOR

Control Unit:



Laboratory 5: AN ENHANCED PROCESSOR

FSM:



STIMULATION ON KIT:

It will create a loop for counting from 0 to 511 and then repeated

LINK FOR THE VIDEO:

https://drive.google.com/file/d/1wzg46x2ZCHYPDFZEh5M2YWxlklEoV28m/view?usp=drive_link



Laboratory 5: AN ENHANCED PROCESSOR

IMAGE FOR ILUSTRATION:

Count = 1



Count = 2



Laboratory 5: AN ENHANCED PROCESSOR

Count = 3



Count = 4



Laboratory 5: AN ENHANCED PROCESSOR

Count = 511



Count = 0

